Linux 外部 RTC
开发指南

版本号: 2.1

发布日期: 2025.08.08

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.07.06	XAA0310	初始版本
2.0	2024.10.31	XAA0331	修正排布格式，重构开发流程，给出编写示例
2.1	2025.08.08	XAA0368	修改文档格式

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 1
1.4.1 标志说明 1
1.4.2 地址与数据描述方法约定 2

2 模块介绍 **3**
2.1 内核配置介绍 3
2.2 dts 配置介绍 4
2.3 驱动代码配置介绍 5
2.3.1 驱动代码开发步骤 5
2.3.2 开发注意事项 5

3 模块接口说明 **6**
3.1 内核态接口 6
3.1.1 sd2059_rtc_read_time 6
3.1.2 sd2059_rtc_set_time 6
3.1.3 sd2059_rtc_read_alarm 7
3.1.4 sd2059_rtc_set_alarm 7
3.1.5 内核态开发示例 7
3.2 用户态接口 11
3.2.1 指令功能描述 11
3.2.2 打开/关闭 RTC 设备 12
3.2.3 设置和获取 RTC 时间 12
3.2.4 用户态开发示例 12

4 外挂 RTC 驱动调试验证 **15**
4.1 I2C 读写 RTC 寄存器 15
4.2 节点测试 15

5 FAQ **17**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

该文档用于介绍 Linux 内核中外挂 RTC 驱动的适配和调试方法，为 RTC 设备的使用者和维护者提供参考。

1.2 目标读者

RTC 驱动及应用层的开发/维护人员。

1.3 适用范围

本文档以 linux-5.15，SD2059 型号 RTC 举例说明，所有内核版本和版型均可参考该文档进行外挂 RTC 配置。

1.4 文档约定

1.4.1 标志说明

注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-1: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

说明

外挂 RTC 驱动开发时：

- **若 RTC 模块厂商已提供驱动代码，则直接适配使用即可。**
- **若无当前型号驱动代码，则可参考类似型号的 RTC 驱动代码，适配调试。**
- **若无任何现存或可参考驱动代码，则需开发新驱动并适配验证。**

为缩短开发时间，开发前尽量先确认上述现有驱动或可参考驱动是否已存在后，再着手开发，本文开发时参考 SD3078 型号驱动代码。

2.1 内核配置介绍

驱动文件位置：${SDK}/bsp/drivers/rtc/sd2059.c

${SDK}/bsp/drivers/rtc/Makefile 修改如下：

```
obj-$(CONFIG_RTC_DRV_RV8803)     += rtc-rv8803.o
obj-$(CONFIG_RTC_DRV_RX4581)     += rtc-rx4581.o
obj-$(CONFIG_RTC_DRV_RX6110)     += rtc-rx6110.o
obj-$(CONFIG_RTC_DRV_RX8010)     += rtc-rx8010.o
obj-$(CONFIG_RTC_DRV_RX8025)     += rtc-rx8025.o
obj-$(CONFIG_RTC_DRV_RX8581)     += rtc-rx8581.o
obj-$(CONFIG_RTC_DRV_S35390A)    += rtc-s35390a.o
obj-$(CONFIG_RTC_DRV_S3C)        += rtc-s3c.o
obj-$(CONFIG_RTC_DRV_S5M)        += rtc-s5m.o
obj-$(CONFIG_RTC_DRV_SA1100)     += rtc-sa1100.o
obj-$(CONFIG_RTC_DRV_SC27XX)     += rtc-sc27xx.o
obj-$(CONFIG_RTC_DRV_SD3078)    += rtc-sd3078.o
obj-$(CONFIG_RTC_DRV_SD2059)    += rtc-sd2059.o
obj-$(CONFIG_RTC_DRV_SH)         += rtc-sh.o
obj-$(CONFIG_RTC_DRV_SNVS)       += rtc-snvs.o
obj-$(CONFIG_RTC_DRV_SPEAR)      += rtc-spear.o
obj-$(CONFIG_RTC_DRV_STARFIRE)   += rtc-starfire.o
obj-$(CONFIG_RTC_DRV_STK17TA8)   += rtc-stk17ta8.o
obj-$(CONFIG_RTC_DRV_ST_LPC)     += rtc-st-lpc.o
obj-$(CONFIG_RTC_DRV_STM32)      += rtc-stm32.o
obj-$(CONFIG_RTC_DRV_STMP)       += rtc-stmp3xxx.o
```

图 2-1: Makefile 配置

${SDK}/bsp/drivers/rtc/Kconfig 修改如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
712
713 config RTC_DRV_SD3078
714         tristate "ZXW Shenzhen whwave SD3078"
715         select REGMAP_I2C
716         help
717           If you say yes here you get support for the ZXW Shenzhen whwave
718           SD3078 RTC chips.
719
720           This driver can also be built as a module. If so, the module
721           will be called rtc-sd3078
722
723 config RTC_DRV_SD2059
724         tristate "ZXW Shenzhen whwave SD2059"
725         select REGMAP_I2C
726         help
727           If you say yes here you get support for the DH Xian quanzhi
728           SD2059 RTC chips.
729
730           This driver can also be built as a module. If so, the module
731           will be called rtc-sd2059
732
733 endif # I2C
```

图 2-2: Kconfig 配置

以上配置，保证驱动可以正常编译至内核。适配时为避免不必要的错误，可以将**内部 RTC 关闭**。

关闭内部 RTC 方法：在根目录下执行./build.sh menuconfig打开配置选项，关闭 CONFIG_AW_RTC 配置项即可。

2.2 dts 配置介绍

外挂 RTC 通常是通过 TWI 完成数据交互，一般挂载在 TWI 总线上。在此次适配中，SD2059 挂载在 S_TWI2 下（对应 TWI8），对应 TWI 的 dts 节点中配置 RTC 信息如下。

修改 ${SDK}/device/config/chips/t527/configs/demo/board.dts 文件，如下所示：

```
&twi8 {
    clock-frequency = <400000>;
    pinctrl-0 = <&s_twi2_pins_default>;
    pinctrl-1 = <&s_twi2_pins_sleep>;
    pinctrl-names = "default", "sleep";
    /* For stability and backwards compatibility, we recommend setting 'twi_drv_used' to 1 */
    twi_drv_used = <1>;
    twi-supply = <&reg_aldo3>;
    status = "disabled";
    status = "okay";
   /* sd2059配置内容 */
    sd2059@32 {
        compatible = "whwave,sd2059";
        reg = <0x32>;          /* 从设备地址（rtc spec中获取）*/
        #clock-cells = <0>;
        clock-frequency = <32768>;   /* 时钟频率 */
        clock-output-name = "xin32k"; /* 时钟名 */
        irq-gpios = <&r_pio PL 2 GPIO_ACTIVE_LOW>; /* 闹钟中断引脚配置 */
        wakeup-source;         /* 表示RTC是具备休眠唤醒能力的中断唤醒源 */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        status = "okay";
    };
};
```

2.3 驱动代码配置介绍

2.3.1 驱动代码开发步骤

在驱动文件中，代码开发主要分为以下 3 个步骤：

1. 驱动框架开发。
2. 读写时间、闹钟函数开发。
3. 闹钟中断函数开发。

说明

驱动框架可借鉴厂家提供的 sd3078 型号 RTC 驱动代码。

2.3.2 开发注意事项

读写时间、读写闹钟函数适配时应先看 spec 熟悉 IP 原理，根据原理进行适配。如：

（1）有的 RTC hour 寄存器只支持 24 进制，有的 12 进制与 24 进制均支持。

（2）有的 RTC 寄存器均以 BCD 码形式保存，有的 week 寄存器不支持 BCD 码形式。

（3）有的 RTC 闹钟可设置为年月日，有的只支持设置时分秒或星期时分秒。

（4）有的 RTC 闹钟可精确到秒，有的只支持到分种。

（5）有的 RTC 闹钟来临时引脚输出电不需配置，有的需要配置。

（6）有的 RTC 写操作会有安全保护操作，有的没有。

除以上内容适配时需要注意外，各厂家 RTC 还会有很多细微差别，请根据具体硬件设计进行适配。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

3.1 内核态接口

3.1.1 sd2059_rtc_read_time

- 函数原型：int sd2059_rtc_read_time(struct device *dev, struct rtc_time *tm)
- 作用：读取硬件时间
- 参数：
  - 参数 1: 设备结构体
  - 参数 2: 存放时间结构体
- 返回：
  - 0: 成功
  - 非 0: 失败

3.1.2 sd2059_rtc_set_time

- 函数原型：int sd2059_rtc_set_time(struct device *dev, struct rtc_time *tm)
- 作用：设置硬件时间
- 参数：
  - 参数 1: 设备结构体
  - 参数 2: 存放时间结构体
- 返回：
  - 0: 成功
  - 非 0: 失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.1.3 sd2059_rtc_read_alarm

- 函数原型：int sd2059_rtc_read_alarm(struct device *dev, struct rtc_wkalrm *alm)
- 作用：读取闹钟时间
- 参数：
  - 参数 1: 设备结构体
  - 参数 2: 存放时间结构体
- 返回：
  - 0: 成功
  - 非 0: 失败

3.1.4 sd2059_rtc_set_alarm

- 函数原型：int sd2059_rtc_set_alarm(struct device *dev, struct rtc_wkalrm *alm)
- 作用：设置闹钟时间
- 参数：
  - 参数 1: 设备结构体
  - 参数 2: 存放时间结构体
- 返回：
  - 0: 成功
  - 非 0: 失败

3.1.5 内核态开发示例

```
static int sd2059_rtc_read_time(struct device *dev, struct rtc_time *tm)
{
 unsigned char hour;

 unsigned char rtc_data[NUM_TIME_REGS] = {0};

 struct i2c_client *client = to_i2c_client(dev);

 struct sd2059 *sd2059 = i2c_get_clientdata(client);

 int ret;

 ret = regmap_bulk_read(sd2059->regmap, SD2059_REG_SC, rtc_data,

    NUM_TIME_REGS);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  if (ret < 0) {
    dev_err(dev, "use I2C reading time from RTC failed with err:%d\n", ret);
    return ret;
  }
  /* Date and time data are stored in registers in BCD format, and conversion is required when reading them */
  tm->tm_sec  = bcd2bin(rtc_data[SD2059_REG_SC] & SEC_MASK);
  tm->tm_min  = bcd2bin(rtc_data[SD2059_REG_MN] & MIN_MASK);
  /*
   \* The sd2059 supports 12/24 hour mode.
   \* When getting time,
   \* we need to convert the 12 hour mode to the 24 hour mode.
   */
  hour = rtc_data[SD2059_REG_HR];
  if (hour & 0x80) /* 24H MODE */
    tm->tm_hour = bcd2bin(rtc_data[SD2059_REG_HR] & HOUR_24H_MASK);
  else if (hour & 0x20) /* 12H MODE PM */
    tm->tm_hour = bcd2bin(rtc_data[SD2059_REG_HR] & HOUR_12H_MASK) + HOUR_12H_DEVIATION;
  else /* 12H MODE AM */
    tm->tm_hour = bcd2bin(rtc_data[SD2059_REG_HR] & HOUR_12H_MASK);
  tm->tm_wday = rtc_data[SD2059_REG_DW] & WEEK_MASK;
  tm->tm_mday = bcd2bin(rtc_data[SD2059_REG_DM] & DAY_MASK);
  tm->tm_mon  = bcd2bin(rtc_data[SD2059_REG_MO] & MON_MASK) - MON_DEVIATION;
  tm->tm_year = bcd2bin(rtc_data[SD2059_REG_YR]) + YEAR_DEVIATION;
  return 0;
}
static int sd2059_rtc_set_time(struct device *dev, struct rtc_time *tm)
{
  unsigned char rtc_data[NUM_TIME_REGS];
  struct i2c_client *client = to_i2c_client(dev);
  struct sd2059 *sd2059 = i2c_get_clientdata(client);
  int ret;
  rtc_data[SD2059_REG_SC] = bin2bcd(tm->tm_sec);
  rtc_data[SD2059_REG_MN] = bin2bcd(tm->tm_min);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    rtc_data[SD2059_REG_HR] = bin2bcd(tm->tm_hour) | 0x80;
    rtc_data[SD2059_REG_DW] = tm->tm_wday & WEEK_MASK;
    rtc_data[SD2059_REG_DM] = bin2bcd(tm->tm_mday);
    rtc_data[SD2059_REG_MO] = bin2bcd(tm->tm_mon + MON_DEVIATION);
    rtc_data[SD2059_REG_YR] = bin2bcd(tm->tm_year - YEAR_DEVIATION);

#if WRITE_PROTECT_EN
    sd2059_enable_reg_write(sd2059);
#endif

    ret = regmap_bulk_write(sd2059->regmap, SD2059_REG_SC, rtc_data,
            NUM_TIME_REGS);
    if (ret < 0) {
        dev_err(dev, "use I2C writing to RTC failed with err:%d\n", ret);
        return ret;
    }

#if WRITE_PROTECT_EN
    sd2059_disable_reg_write(sd2059);
#endif

    return 0;
}

static int sd2059_rtc_read_alarm(struct device *dev, struct rtc_wkalrm *alm)
{
    unsigned char rtc_data[NUM_ALARM_REGS] = {0};
    struct i2c_client *client = to_i2c_client(dev);
    struct sd2059 *sd2059 = i2c_get_clientdata(client);
    int ret;

    ret = regmap_bulk_read(sd2059->regmap, SD2059_REG_ALARM_SC, rtc_data,
            NUM_ALARM_REGS);
    if (ret < 0) {
        dev_err(dev, "use I2C reading alarm from RTC failed with err:%d\n", ret);
        return ret;
    }

    alm->time.tm_sec  = bcd2bin(rtc_data[SD2059_REG_ALARM_SC - SD2059_REG_ALARM_SC]);
    alm->time.tm_min  = bcd2bin(rtc_data[SD2059_REG_ALARM_MN - SD2059_REG_ALARM_SC]);
    alm->time.tm_hour = bcd2bin(rtc_data[SD2059_REG_ALARM_HR - SD2059_REG_ALARM_SC]);
    alm->time.tm_mday = bcd2bin(rtc_data[SD2059_REG_ALARM_DW - SD2059_REG_ALARM_SC]);
    alm->time.tm_wday = bcd2bin(rtc_data[SD2059_REG_ALARM_DM - SD2059_REG_ALARM_SC]);
    alm->time.tm_mon  = bcd2bin(rtc_data[SD2059_REG_ALARM_MO - SD2059_REG_ALARM_SC]);
    alm->time.tm_year = bcd2bin(rtc_data[SD2059_REG_ALARM_YR - SD2059_REG_ALARM_SC]);

    /* check whether alarm interruption is allowed */
    if ((rtc_data[SD2059_ALARM_ALLOED - SD2059_REG_ALARM_SC] & 0x7f) != 0)  /* SD2059_ALARM_ALLOED register are
        only valid for the lower seven bits */
        alm->enabled = 1;

    /* check if there is an alarm interrupt */
    ret = regmap_bulk_read(sd2059->regmap, SD2059_REG_CTRL1, rtc_data, 1);
    if (ret < 0) {
        dev_err(dev, "use I2C reading SD2059_REG_CTRL1 from RTC failed with err:%d\n", ret);
        return ret;
    }
    if ((rtc_data[0] & SD2059_INT_AF) != 0)
        alm->pending = 1;
    else
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    alm->pending = 0;

    return 0;
}

static int sd2059_rtc_set_alarm(struct device *dev, struct rtc_wkalrm *alm)
{
    unsigned char rtc_data[NUM_ALARM_REGS];
    struct i2c_client *client = to_i2c_client(dev);
    struct sd2059 *sd2059 = i2c_get_clientdata(client);
    int ret;

    rtc_data[SD2059_REG_ALARM_SC - SD2059_REG_ALARM_SC] = bin2bcd(alm->time.tm_sec);
    rtc_data[SD2059_REG_ALARM_MN - SD2059_REG_ALARM_SC] = bin2bcd(alm->time.tm_min);
    rtc_data[SD2059_REG_ALARM_HR - SD2059_REG_ALARM_SC] = bin2bcd(alm->time.tm_hour);
    rtc_data[SD2059_REG_ALARM_DW - SD2059_REG_ALARM_SC] = bin2bcd(alm->time.tm_wday);
    rtc_data[SD2059_REG_ALARM_DM - SD2059_REG_ALARM_SC] = bin2bcd(alm->time.tm_mday);
    rtc_data[SD2059_REG_ALARM_MO - SD2059_REG_ALARM_SC] = bin2bcd(alm->time.tm_mon + MON_DEVIATION);
    rtc_data[SD2059_REG_ALARM_YR - SD2059_REG_ALARM_SC] = bin2bcd(alm->time.tm_year - YEAR_DEVIATION);

    if (alm->enabled == 1)
        rtc_data[SD2059_ALARM_ALLOED - SD2059_REG_ALARM_SC] = 0x7f; /* SD2059_ALARM_ALLOED register are only
            valid for the lower seven bits */
    else
        rtc_data[SD2059_ALARM_ALLOED - SD2059_REG_ALARM_SC] = 0;

#if WRITE_PROTECT_EN
    sd2059_enable_reg_write(sd2059);
#endif

    ret = regmap_bulk_write(sd2059->regmap, SD2059_REG_ALARM_SC, rtc_data,
            NUM_ALARM_REGS);
    if (ret < 0) {
        dev_err(dev, "writing alarm from RTC failed with err:%d\n", ret);
        return ret;
    }

    ret = regmap_bulk_read(sd2059->regmap, SD2059_REG_CTRL2, rtc_data, 1);
    rtc_data[0] &= ~SD2059_INTS1;
    rtc_data[0] |= SD2059_INTS0;
    rtc_data[0] |= SD2059_INTAE;
    rtc_data[0] &= ~SD2059_IM;

    /* set enable interrupt, set alarm interrupt output, set single event alarm */
    ret = regmap_bulk_write(sd2059->regmap, SD2059_REG_CTRL2, rtc_data, 1);
    if (ret < 0) {
        dev_err(dev, "writing alarm from RTC failed with err:%d\n", ret);
        return ret;
    }

#if WRITE_PROTECT_EN
    sd2059_disable_reg_write(sd2059);
#endif

    return 0;
}

static irqreturn_t rtc_handle_irq(int irq, void *dev_id)
{
    struct i2c_client *client = dev_id;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    struct sd2059 *sd2059 = i2c_get_clientdata(client);
    unsigned char rtc_data[1];

    /* clear rtc interrupt flag bit */
    regmap_bulk_read(sd2059->regmap, SD2059_REG_CTRL1, rtc_data, 1);
    rtc_data[0] &= ~SD2059_INT_AF;
    regmap_bulk_write(sd2059->regmap, SD2059_REG_CTRL1, rtc_data, 1);

    rtc_update_irq(sd2059->rtc, 1, RTC_IRQF | RTC_AF);

    return IRQ_HANDLED;
}

static const struct rtc_class_ops sd2059_rtc_ops = {
    .read_time  = sd2059_rtc_read_time,
    .set_time   = sd2059_rtc_set_time,
    .read_alarm = sd2059_rtc_read_alarm,
    .set_alarm  = sd2059_rtc_set_alarm,
};
```

3.2 用户态接口

3.2.1 指令功能描述

表 3-1: RTC 驱动 ioctl 指令功能描述

指令	功能描述
RTC_ALM_READ	读取闹钟时间
RTC_ALM_SET	设置闹钟时间
RTC_SET_TIME	设置时间与日期
RTC_PIE_ON	开 RTC 全局中断
RTC_PIE_OFF	关 RTC 全局中断
RTC_AIE_ON	使能 RTC 闹钟中断
RTC_AIE_OFF	禁止 RTC 闹钟中断
RTC_UIE_ON	使能 RTC 更新中断
RTC_UIE_OFF	禁止 RTC 更新中断
RTC_IRQP_SET	设置中断的频率

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.2 打开/关闭 RTC 设备

使用标准的文件打开函数：

```
int open(const char *pathname, int flags);
int close(int fd);
```

需要引用头文件：

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
```

3.2.3 设置和获取 RTC 时间

同样使用标准的ioctl函数：

```
int ioctl(int d, int request, ...);
```

需要引用头文件：

```
#include <sys/ioctl.h>
#include <linux/rtc.h>
```

设置 RTC 时间：

```
ioctl(fd, RTC_SET_TIME, &rtc_tm)
```

获取 RTC 时间：

```
ioctl(fd, RTC_RD_TIME, &rtc_tm)
```

3.2.4 用户态开发示例

此demo程序是打开一个RTC设备，然后设置和获取RTC时间以及设置闹钟功能。

```
#include <stdio.h>     /*标准输入输出定义*/
#include <stdlib.h>    /*标准函数库定义*/
#include <unistd.h>    /*Unix 标准函数定义*/
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>     /*文件控制定义*/
#include <linux/rtc.h> /*RTC支持的CMD*/
#include <errno.h>     /*错误号定义*/
#include <string.h>

#define RTC_DEVICE_NAME   "/dev/rtc0"

int set_rtc_timer(int fd)
{
  struct rtc_time rtc_tm = {0};
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  struct rtc_time rtc_tm_temp = {0};

  rtc_tm.tm_year = 2020 - 1900;  /* 需要设置的年份，需要减1900 */
  rtc_tm.tm_mon = 11 - 1;       /* 需要设置的月份,需要确保在0-11范围 */
  rtc_tm.tm_mday = 21;     /* 需要设置的日期 */
  rtc_tm.tm_hour = 10;     /* 需要设置的时间 */
  rtc_tm.tm_min = 12;      /* 需要设置的分钟时间 */
  rtc_tm.tm_sec = 30;      /* 需要设置的秒数 */

  /* 设置RTC时间 */
  if (ioctl(fd, RTC_SET_TIME, &rtc_tm) < 0) {
    printf("RTC_SET_TIME failed\n");
    return -1;
  }

  /* 获取RTC时间 */
  if (ioctl(fd, RTC_RD_TIME, &rtc_tm_temp) < 0) {
    printf("RTC_RD_TIME failed\n");
    return -1;
  }
  printf("RTC_RD_TIME return %04d-%02d-%02d %02d:%02d:%02d\n",
        rtc_tm_temp.tm_year + 1900, rtc_tm_temp.tm_mon + 1, rtc_tm_temp.tm_mday,
        rtc_tm_temp.tm_hour, rtc_tm_temp.tm_min, rtc_tm_temp.tm_sec);
  return 0;
}

int set_rtc_alarm(int fd)
{
  struct rtc_time rtc_tm = {0};
  struct rtc_time rtc_tm_temp = {0};

  rtc_tm.tm_year = 0;  /* 闹钟忽略年设置 */
  rtc_tm.tm_mon = 0;   /* 闹钟忽略月设置 */
  rtc_tm.tm_mday = 0;  /* 闹钟忽略日期设置 */
  rtc_tm.tm_hour = 10; /* 需要设置的时间 */
  rtc_tm.tm_min = 12;  /* 需要设置的分钟时间 */
  rtc_tm.tm_sec = 30;  /* 需要设置的秒数 */

  /* set alarm time */
  if (ioctl(fd, RTC_ALM_SET, &rtc_tm) < 0) {
    printf("RTC_ALM_SET failed\n");
    return -1;
  }

  if (ioctl(fd, RTC_AIE_ON) < 0) {
    printf("RTC_AIE_ON failed!\n");
    return -1;
  }

  if (ioctl(fd, RTC_ALM_READ, &rtc_tm_temp) < 0) {
    printf("RTC_ALM_READ failed\n");
    return -1;
  }

  printf("RTC_ALM_READ return %04d-%02d-%02d %02d:%02d:%02d\n",
          rtc_tm_temp.tm_year + 1900, rtc_tm_temp.tm_mon + 1, rtc_tm_temp.tm_mday,
          rtc_tm_temp.tm_hour, rtc_tm_temp.tm_min, rtc_tm_temp.tm_sec);
  return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
int main(int argc, char *argv[])
{
  int fd;
  int ret;

  /* open rtc device */
  fd = open(RTC_DEVICE_NAME, O_RDWR);
  if (fd < 0) {
    printf("open rtc device %s failed\n", RTC_DEVICE_NAME);
    return -ENODEV;
  }

 /* 设置RTC时间 */
 ret = set_rtc_timer(fd);
 if (ret < 0) {
   printf("set rtc timer error\n");
   return -EINVAL;
 }

 /* 设置闹钟 */
 ret = set_rtc_alarm(fd);
 if (ret < 0) {
   printf("set rtc alarm error\n");
   return -EINVAL;
 }

 close(fd);
 return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 外挂 RTC 驱动调试验证

RTC 模块主要涉及 4 个功能：读时间、写时间、读闹钟、写闹钟，这 4 个功能使用 4 个函数实现。以下调试方法可以结合使用，达到目的即可。

4.1 I2C 读写 RTC 寄存器

外挂 RTC（sd2059）通常使用 I2C 通信协议与主处理器进行通信，在调试过程中可以使用 TWI 提供指令直接读写 RTC 寄存器定位问题，其指令如下：

- 读取一个寄存器指令：i2cget -f -y 8 0x32 0x00。

8 代表 TWI8 下挂载的从设备，0x32 代表从设备地址，0x00 表示读取设备寄存器地址。

- 写入一个寄存器指令：i2cset -f -y 8 0x32 0x01 0x03。

8 代表 TWI8 下挂载的从设备，0x32 代表从设备地址，0x01 表示写入设备寄存器地址；0x03 代表写入寄存器值。

使用上述指令可以观测外挂 RTC 寄存器状态，实现调试功能。

4.2 节点测试

小机端执行命令:

```
ls /dev/rtc*
```

能够查询到 RTC 设备，表明内核 RTC 驱动正确加载。

内核框架为 RTC 设备提供了 sysfs 节点，方便使用与调试。外挂 RTC 注册进内核后，可通过操作节点调用 rtc 内部读写时间以及读写闹钟的函数。

- 设置当前时间：date -s "2008-05-12 14:28:04"。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

将当前时间写入硬件时钟：hwclock -w。

一般前两条指令连续使用将设置时间写入 rtc。

- 查看时间：cat /sys/class/rtc/rtc0/time；即可显示当前时间（时分秒）。
- 查看日期：cat /sys/class/rtc/rtc0/date；即可显示当前日期（年月日）。
- 设置闹钟：echo "+10" > sys/class/rtc/rtc0/wakealarm；表示设置当前时间 10 秒后产生闹钟；此句指令是读取当前 rtc 时间加上延迟时间后写入 rtc 闹钟时间实现功能。
- 查看闹钟：cat /sys/class/rtc/rtc0/wakealarm；表示查看当前闹钟时间，其结果是一个时间戳，表示从 1970 年 1 月 1 日开始的秒数, 转换为真正的时间使用 date -d 命令即可。

如:

```
/# cat sys/class/rtc/rtc0/wakealarm   /# 950594921   /# date -d @950594921   /# Tue Feb 15 06:08:41 UTC 2000
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 FAQ

- BCD 码是二进制编码的一种形式，用于表示十进制数，它将每个十进制数表示为 4 位二进制数。例如：54 可表示为 01010100。BCD 码一般用于 RTC 芯片。
- RTC 类似型号驱动可能有差别，比如 SD3078 型号 RTC 的 alarm 寄存器读写是不需要进行 BCD 码转化的，而 SD2059 型号 RTC 的 alarm 寄存器需要进行 BCD 码转化。一般读写时间以及读写闹钟是否需要 BCD 码转化要看 data sheet 介绍（大部分都需要）。
- 如下图是内核中月份与年份的定义：月份的范围是 0-11；年份的起始范围是 1900。

这表示：在通过节点向 RTC 写入时间时，月份变量 tm_mon 需要进行加 1 操作，才能真正表示当前月份。如果设置时间为 2002 年，那么 tm_year 等于 102；若 RTC 年份也是从 1900 年开始便可直接写入 102；sd2059 是从 2000 年开始，则需要减 100 后写入。在调试时需要根据具体情况具体分析！

```
103 struct tm {
104         /*
105          * the number of seconds after the minute, normally in the range
106          * 0 to 59, but can be up to 60 to allow for leap seconds
107          */
108         int tm_sec;
109         /* the number of minutes after the hour, in the range 0 to 59*/
110         int tm_min;
111         /* the number of hours past midnight, in the range 0 to 23 */
112         int tm_hour;
113         /* the day of the month, in the range 1 to 31 */
114         int tm_mday;
115         /* the number of months since January, in the range 0 to 11 */
116         int tm_mon;
117         /* the number of years since 1900 */
118         long tm_year;
119         /* the number of days since Sunday, in the range 0 to 6 */
120         int tm_wday;
121         /* the number of days since January 1, in the range 0 to 365 */
122         int tm_yday;
123 };
124
```

图 5-1: 内核日期定义差异

- 内核中，rtc_time 结构体有两个 day 相关变量：tm_mday 以及 tm_yday。其中 tm_mday 表示正常日期，tm_yday 表示一年中第多少天。在读写时间以及读写闹钟时需要注意。
- RTC 核心功能就是实现四个函数，但是开发调试过程中要注意每个 IP 的硬件设计是如何实现的（尤其是闹钟产生中断的流程及配置）。在调试时推荐先根据 ip 设计裸写寄存器验证功能，再根据逻辑书写代码。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 闹钟中断中出现 panic：RTC 闹钟中断会将引脚 GPIO 电平拉低进入中断函数，中断函数中需要通过 TWI 将 RTC 寄存器中中断标志位清 0，而：

（1）TWI 完成读写操作依靠中断完成，Linux 不支持中断嵌套。

（2）TWI 传输过程中会睡眠，等待信号量。中断中不能睡眠。

解决办法是使用 devm_request_threaded_irq() 函数注册中断函数，将中断线程化。

参考代码路径：${SDK}/bsp/drivers/rtc/sd2059.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。